

✓ Clase 22 — Primer contacto con Colab

Diplomado IA Aplicada al Diseño · UDD · 2026 · Prof. Darío Osorio

Este notebook es tu primera experiencia con código de IA. **No necesitas saber programar** — solo hay que ejecutar las celdas (botón Play o `Shift + Enter`) y donde diga "TU TURNO", modificar una lista o un texto.

Cómo se usa este notebook

1. Ejecuta las celdas **en orden**, de arriba hacia abajo.
2. La primera celda instala lo necesario (tarda ~1 minuto la primera vez).
3. Cuando aparezca `# TU TURNO`, editas ese pedazo con datos de tu proyecto.
4. Guarda una copia en tu Drive: `Archivo → Guardar copia en Drive`.

✓ Paso 0 — Instalar las herramientas (una sola vez)

```
!pip install -q sentence-transformers transformers matplotlib
print("Listo, todo instalado.")
```

```
Listo, todo instalado.
```

✓ Ejercicio 1 — Ver embeddings de tu propio proyecto

Los embeddings son la forma en que la IA "entiende" palabras: las convierte en coordenadas matemáticas donde los conceptos parecidos quedan cerca.

Vamos a comprobarlo con palabras de tu proyecto.

```
from sentence_transformers import SentenceTransformer, util
# Modelo pequeño, se descarga la primera vez (30 seg aprox)
modelo = SentenceTransformer('sentence-transformers/all-MiniLM-L6-v2')
print("Modelo cargado.")
```

```
Warning: You are sending unauthenticated requests to the
WARNING:huggingface_hub.utils._http:WARNING:
modules.json: 100% 349/349 [00:00<00:00, 12.8kB/s]
config_sentence_transformers.json: 100% 116/116 [00:00<00:00, 11.6kB/s]
README.md: 100% 10.5k/10.5k [00:00<00:00, 615kB/s]
sentence_bert_config.json: 100% 53.0/53.0 [00:00<00:00, 53.0kB/s]
config.json: 100% 612/612 [00:00<00:00, 43.3kB/s]
model.safetensors: reconstructing file: 100% 90.9MB [00:00<00:00, 90.9MB/s]
model.safetensors: downloading bytes: 85.0MB, 8.1MB/s [00:00<00:00, 8.1MB/s]
Loading weights: 100% 103/103 [00:00<00:00, 2924kB/s]
tokenizer_config.json: 100% 350/350 [00:00<00:00, 350kB/s]
vocab.txt: 100% 232k/232k [00:00<00:00, 6.24MB/s]
tokenizer.json: 100% 466k/466k [00:00<00:00, 18.0MB/s]
```

✓ TU TURNO — modifica esta lista con palabras de tu proyecto

```
# Reemplaza estas palabras con las de tu propio proyecto
# Mezcla conceptos cercanos y lejanos para ver la geometria
palabras = [
    # cluster diseno
    "sobremesa", "conversacion", "familia", "celular",
    # cluster IA
    "almuerzo", "comida", "pantalla", "desconexion",
    # cluster usuario
    "reunion", "experiencia", "comunicacion",
]

# Comparar cada palabra con las demas
print("MATRIZ DE SIMILITUD (cuanto mas alto, mas parecidos):\n")
print(f"{'':15s}", end="")
for p in palabras[:6]:
    print(f"{p[:8]:>10s}", end="")
    print()
for p1 in palabras[:6]:
    print(f"{p1[:14]:15s}", end="")
    for p2 in palabras[:6]:
        emb1 = modelo.encode(p1, convert_to_tensor=True)
        emb2 = modelo.encode(p2, convert_to_tensor=True)
        sim = util.cos_sim(emb1, emb2).item()
        print(f"{sim:>10.2f}", end="")
    print()
```

MATRIZ DE SIMILITUD (cuanto mas alto, mas parecidos):

	sobremes	conversa	familia	celular	almuerzo	comida
sobremesa	1.00	0.30	0.19	0.31	0.40	0.33
conversacion	0.30	1.00	0.27	0.25	0.27	0.34
familia	0.19	0.27	1.00	0.17	0.19	0.27
celular	0.31	0.25	0.17	1.00	0.27	0.42
almuerzo	0.40	0.27	0.19	0.27	1.00	0.22
comida	0.33	0.34	0.27	0.42	0.22	1.00

Lo que estás viendo: cada celda muestra qué tan "parecidas" son dos palabras para el modelo. Un valor cercano a 1.00 = casi lo mismo. Cercano a 0 = distintos. Negativo = opuestos.

Anota en tu bitácora: ¿qué combinación te sorprendió más? ¿alguna pareció injusta o rara?

✓ Ejercicio 2 — Modelo analítico (clasificador de sentimiento)

Este modelo NO genera nada. Solo clasifica: le das un texto, te dice si es positivo o negativo.

```
from transformers import pipeline
clasificador = pipeline("sentiment-analysis",
                        model="nlptown/bert-base-multilingual-uncased-sentiment")
print("Clasificador listo.")
```

config.json: 100%	953/953 [00:00<00:00, 16.4kB/s]
model.safetensors: reconstructing file: 100%	669MB / 669MB, 56.2MB/s
model.safetensors: downloading bytes:	634MB, 48.9MB/s
Loading weights: 100%	201/201 [00:00<00:00, 3608.86it/s]
tokenizer_config.json: 100%	39.0/39.0 [00:00<00:00, 1.72kB/s]
vocab.txt: 100%	872k/872k [00:00<00:00, 12.3MB/s]
special_tokens_map.json: 100%	112/112 [00:00<00:00, 6.24kB/s]
Clasificador listo.	

✓ TU TURNO — prueba con frases de tu proyecto o inventadas

```
frases_a_clasificar = [
    "Este diseno me parece extraordinario, sobrio y memorable.",
    "La interfaz es completamente confusa, no sirve para nada.",
    "El proyecto cumple con lo esperado pero no destaca.",
    # Agrega frases del contexto de tu proyecto:
    "TU FRASE 1",
    "TU FRASE 2",
]

for frase in frases_a_clasificar:
    r = clasificador(frase)[0]
    print(f"  {r['label']} ({r['score']:.0%}) -> '{frase[:60]}'")
```

Observa: cada frase recibe una etiqueta (1-5 estrellas) y una probabilidad. Fíjate que las respuestas son **iguales cada vez** que ejecutes la celda. Esto es característico de los modelos analíticos: son deterministas.

✓ Ejercicio 3 — Modelo generativo (produce texto nuevo)

Ahora al revés: le damos un inicio de frase y el modelo la continua.

```
generador = pipeline("text-generation",
                      model="datificarte/gpt2-small-spanish",
                      max_new_tokens=60)
print("Generador listo.")
```

✓ TU TURNO — prueba con un prompt de tu proyecto

```
prompt = "El diseno contemporaneo en Latinoamerica enfrenta el desafio de"

# Ejecuta esta celda VARIAS veces y ve como cambia la respuesta
resultado = generador(prompt, num_return_sequences=1,
                      do_sample=True, temperature=0.8)
print(resultado[0]['generated_text'])
```

Discusión para la sala breakout:

- ¿Qué diferencia hay entre las respuestas del clasificador (mismas cada vez) y las del generador (distintas cada vez)?
- ¿Cuál te sería más útil para tu proyecto de diseño?
- ¿Alguna vez el generador dijo algo "raro" o inventado? Los modelos generativos alucinan — es parte de cómo funcionan.

Cierre del notebook

Al terminar, guarda una copia:

1. Archivo → Guardar copia en Drive.
2. Renombrala: Clase_22_TuNombre.ipynb.
3. Descárgala: Archivo → Descargar → .ipynb.
4. Súbela a tu repositorio de GitHub (guía en Clase_22_guia_estudiante.md).

Pregunta para tu bitácora

¿Qué te pareció más raro, incomodo o interesante de ver el "motor" de la IA por dentro? ¿Cambió algo en cómo piensas usar estas herramientas?

